Question 1: Object-Oriented Programming (OOP) [8 marks]

Introduction: (2 marks)

Object-Oriented Programming (OOP) is a programming paradigm that structures software design around objects rather than functions and logic. An object is a self-contained entity that encapsulates both state (data/attributes) and behaviour (methods/operations). OOP was developed to address the limitations of procedural programming in managing large, complex software systems. Languages like Java, C++, Python, C#, and

Core Concepts: (3 marks)

OOP is founded on four core pillars. Encapsulation bundles data and methods within a class and restricts direct external access to internal state using access modifiers (private, protected, public), exposing only a controlled interface through getter/setter methods · this prevents unintended data corruption. Abstraction hides complex implementation details and exposes only the essential interface through abstract classes and interfaces, reducing cognitive load for developers. Inheritance allows a child class to acquire attributes and methods from a parent class, enabling code reuse and establishing IS-A hierarchical relationships · for example, Car IS-A Vehicle. Multiple inheritance (supported in C++ but not Java) allows a class to inherit

Examples: (2 marks)

A BankAccount class encapsulates balance (private) and exposes deposit(amount) and withdraw(amount) methods · Encapsulation. SavingsAccount extends BankAccount and overrides calculateInterest() to add 4% per annum · Inheritance and Polymorphism. An abstract class Shape declares abstract method area(); Circle and Rectangle override it with specific formulas · Abstraction. In a Vehicle hierarchy: move() is called on a Vehicle reference pointing to a Car or Truck object; at runtime,

Conclusion: (1 mark)

OOP provides a robust, scalable approach to software development by modelling real-world entities as objects. Its four pillars collectively reduce code duplication, increase maintainability, and support team collaboration through clear module boundaries, making OOP

Question 2: Database Normalization [8 marks]

Introduction: (2 marks)

Database normalization is a systematic process of organizing a relational database schema to minimize data redundancy and eliminate update, insert, and delete anomalies. It involves decomposing larger tables into smaller, well-structured tables and defining relationships between them using foreign keys. Normalization was formalized by Edgar F. Codd (inventor of the relational model) and follows a series of progressive rules called

Core Concepts: (3 marks)

First Normal Form (1NF): each column must contain atomic (indivisible) values, every row must be unique (primary key exists), and there must be no repeating groups or arrays within a column. Second Normal Form (2NF): the table must be in 1NF and every non-prime attribute must be fully functionally dependent on the entire primary key · eliminates partial dependencies (applicable when the key is composite). Third Normal Form (3NF): the table must be in 2NF and no non-prime attribute may be transitively dependent on the primary key via another non-prime attribute. Boyce-Codd Normal Form

Examples: (2 marks)

Consider Student_Course(StudentID, StudentName, CourseID, CourseName, InstructorName, Grade). Partial dependency: StudentName depends only on StudentID (violates 2NF).

Transitive dependency: InstructorName depends on CourseID, not directly on the composite key. Decompose to: Student(StudentID PK, StudentName), Course(CourseID PK, CourseName, InstructorID FK), Instructor(InstructorID PK, InstructorName), Enrollment(StudentID FK, CourseID FK, Grade). This eliminates: Update anomaly (changing

Conclusion: (1 mark)

Normalization produces clean, anomaly-free database schemas by systematically removing redundancy. While 3NF or BCNF is sufficient for most OLTP systems, data warehouses may intentionally denormalize for query performance.

Question 3: Operating System Process Management [8 marks]

Introduction: (2 marks)

Process management is a core responsibility of the operating system (OS), which must create, schedule, synchronize, and terminate processes while ensuring CPU utilization, fairness, and system stability. A process is an instance of a program in execution, distinct from the program itself (a passive entity stored on disk). Each process has its own address space (code, data, heap, stack segments), program counter, CPU registers, and Process Control

Core Concepts: (3 marks)

Process States: New · Ready · Running · Waiting/Blocked · Terminated. A process moves between states based on scheduling decisions and I/O events. PCB stores: PID, state, program counter, CPU registers, memory limits, open files list, accounting info, I/O status, and scheduling priority. Context switching saves the current PCB and loads the next process's PCB · incurs overhead (~1-2 µs on modern hardware). CPU Scheduling Algorithms · FCFS (First-Come First-Served): non-preemptive, simple, convoy effect. SJF (Shortest Job First): optimal average waiting time, requires future knowledge; SRTF is its preemptive variant. Round Robin (RR): preemptive,

Examples: (2 marks)

Round Robin with quantum=4ms on P1(24ms), P2(3ms), P3(3ms): execution order P1(4), P2(3), P3(3), P1(4)×5. Average waiting time = (6+4+7)/3 = 5.67ms. Context switch example: P1 at quantum expiry · save P1's PCB, load P2's PCB, resume P2 at saved PC. Deadlock example: P1 holds Resource A, waits for B; P2 holds B, waits for A · circular wait. Banker's

Conclusion: (1 mark)

Effective process management is the cornerstone of OS performance. The scheduler must balance CPU utilization, throughput, turnaround time, waiting time, and response time. Modern OSes use MLFQ schedulers with multicore-aware load

Question 4: Computer Networks and the OSI Model [8 marks]

Introduction: (2 marks)

A computer network is a collection of interconnected devices (hosts, routers, switches) that communicate by exchanging data packets according to agreed-upon protocols. Networks range from LANs (room or building), WANs (city or global), MANs (metropolitan), and the Internet (global WAN). The OSI (Open Systems Interconnection) model, standardized by ISO in 1984 (ISO/IEC 7498-1), provides a seven-layer

Core Concepts: (3 marks)

Layer 7 · Application: provides network services directly to end-user applications. Protocols: HTTP/HTTPS (web), SMTP/IMAP/POP3 (email), FTP/SFTP (file transfer), DNS (name

resolution), DHCP (address allocation), SNMP (network management). Layer 6 · Presentation: data translation (encoding/decoding), encryption/decryption (SSL/TLS), compression. Formats: JPEG, MP3, ASCII, EBCDIC. Layer 5 · Session: establishes, manages, and terminates sessions. Handles checkpointing and dialog control (half/full duplex). Layer 4 · Transport: end-to-end communication, segmentation, flow control, error recovery, multiplexing. TCP (reliable, connection-oriented, 3-way handshake, SYN-SYN/ACK-ACK, sliding window flow control, congestion control). UDP (unreliable, connectionless, low overhead · DNS,

Examples: (2 marks)

HTTPS request to www.example.com: DNS resolves domain · IP (Application). TLS encrypts data (Presentation). TCP session via 3-way handshake (Session + Transport). IP packet routed across internet (Network). Ethernet frame with MAC addresses traverses LAN (Data Link). Electrical/optical/radio signals transmitted (Physical). Each layer adds a header (encapsulation) on send; strips it (decapsulation)

Conclusion: (1 mark)

The OSI model is the definitive reference for understanding, designing, and troubleshooting network systems. The practical Internet uses the TCP/IP model (4 layers: Application, Transport, Internet, Network Access), but

Question 5: Data Structures · Binary Trees and BST [8 marks]

Introduction: (2 marks)

A tree is a non-linear hierarchical data structure composed of nodes connected by edges, with a designated root node and no cycles. A binary tree is a tree where each node has at most two children: left child and right child. Trees are fundamental in computer science, used for representing hierarchical data (file systems, XML/DOM), enabling efficient searching and sorting, and

Core Concepts: (3 marks)

Binary Tree types: Full (every node has 0 or 2 children), Complete (all levels fully filled except possibly the last, which is filled left-to-right), Perfect (all internal nodes have 2 children, all leaves at same level), Degenerate/Skewed (each node has only one child · degenerates to a linked list). Binary Search Tree (BST): a binary tree with the BST property · for every node N, all keys in the left subtree are less than N.key, and all keys in the right subtree are greater. Operations · Search: O(h); Insert: O(h); Delete: O(h); h = height. In-order traversal of BST gives sorted output in O(n). Worst-case

Examples: (2 marks)

BST insertion of [50, 30, 70, 20, 40, 60, 80]: root=50, left=30, right=70, 30's left=20, 30's right=40, 70's left=60, 70's right=80 · perfectly balanced, h=2. In-order traversal: 20, 30, 40, 50, 60, 70, 80 (sorted). BST search for 40: 50·30·40 (2 comparisons). Delete node 30 (two children): replace with in-order successor (40), then

Conclusion: (1 mark)

Binary trees and BSTs provide efficient O(log n) search, insert, and delete operations for ordered data when balanced. Self-balancing variants (AVL, Red-Black) guarantee logarithmic performance regardless of insertion

Question 6: Sorting Algorithms and Algorithm Analysis [8 marks]

Introduction: (2 marks)

Sorting is the process of arranging elements in a defined order (ascending or descending). It is one of the most fundamental operations in computer science, underpinning database query optimization, binary search, data compression, and computational geometry. Algorithm analysis quantifies efficiency using Big-O notation: O(f(n)) describes the upper bound on time or space as input size

Core Concepts: (3 marks)

Bubble Sort: repeatedly swaps adjacent out-of-order elements. Best O(n) (already sorted, with flag), Worst/Average O(n²). Stable, in-place. Selection Sort: finds minimum element and places at front each pass. Always O(n²). Unstable, in-place. Insertion Sort: builds sorted array left-to-right by inserting each element at correct position. Best O(n), Worst O(n²). Stable, in-place. Preferred for small or nearly-sorted arrays. Used in TimSort for small sub-arrays. Merge Sort: divide-and-conquer · split into halves, sort recursively, merge. Always O(n log n). Stable. O(n) space (not in-place). Preferred for linked lists and external sorting. Quick Sort: choose pivot, partition array (elements < pivot

Examples: (2 marks)

Quick Sort on [3,6,8,10,1,2,1] with pivot=1: partition gives [1,1,3,6,8,10,2] · recurse on left/right halves. Merge Sort: [38,27,43,3] · split [38,27] [43,3] · sort [27,38] [3,43] · merge [3,27,38,43]. Counting Sort on [4,2,2,8,3,3,1]: count array indexed

Conclusion: (1 mark)

Choosing the right sorting algorithm depends on input size, data characteristics (nearly sorted, duplicates, key range), stability requirements, and memory constraints. In practice, library sorts (TimSort, introsort) use

Question 7: Software Engineering · SDLC and Agile [8 marks]

Introduction: (2 marks)

Software Engineering applies systematic, disciplined, quantifiable approaches to the development, operation, and maintenance of software · adapting engineering principles to software systems. The Software Development Life Cycle (SDLC) is a framework that defines the phases, activities, and deliverables for producing high-quality software within time and budget constraints. Selecting the right SDLC

Core Concepts: (3 marks)

Classical SDLC Models: Waterfall · sequential phases (Requirements · Design · Implementation · Testing · Deployment · Maintenance); simple but inflexible for changing requirements; suits well-understood, stable domains. V-Model · extends Waterfall with verification and validation at each phase; each dev phase has a corresponding test phase. Iterative Model · develops system in iterations, refining with each cycle. Spiral Model · risk-driven; combines iterative development with systematic risk assessment; 4 quadrants: Determine Objectives, Identify Risks, Develop/Test, Plan Next Iteration. Agile Methodology: values (Agile Manifesto 2001): Individuals & Interactions over processes, Working Software over documentation, Customer Collaboration over

Examples: (2 marks)

Waterfall for avionics software: requirements fully frozen, strict V&V, DO-178C certification requires documented phase gates · Waterfall is appropriate. Scrum for e-commerce platform: Sprint 1: user authentication, Sprint 2: product catalogue, Sprint 3: shopping cart · business can reprioritize backlog between sprints. TDD example: write test assertUser.login(correct_password) == True · implement login() · test passes · refactor

Conclusion: (1 mark)

Modern software engineering favours Agile methodologies for their ability to accommodate changing requirements and deliver value incrementally. However, safety-critical and regulatory-constrained domains still rely on heavyweight models like Waterfall or V-Model. Effective engineers

Question 8: Computer Architecture · CPU and Memory Hierarchy [8 marks]

Introduction: (2 marks)

Computer architecture defines the functional structure and behaviour of a computer system at the ISA (Instruction Set Architecture) level · the interface between hardware and software. The von Neumann architecture (1945) introduced the stored-program concept: program instructions and data reside in the same memory, fetched and executed sequentially. Modern CPUs implement

Core Concepts: (3 marks)

CPU Components: ALU (Arithmetic Logic Unit · performs arithmetic and logical operations), CU (Control Unit · fetches, decodes, and issues micro-operations), Registers (PC/IP, IR, MAR, MDR, accumulator, general-purpose). Instruction Cycle: Fetch (load instruction from memory[PC] into IR) · Decode (CU interprets opcode) · Execute (ALU performs operation) · Store (write result to register/memory). Pipelining: overlaps multiple instruction stages (IF/ID/EX/MEM/WB) to increase throughput. CPI approaches 1 with deep pipeline. Hazards: Data (RAW/WAR/WAW dependencies · solved by forwarding/stalling), Control (branch prediction · 2-bit saturating counter, BTB), Structural (resource conflicts · solved by duplication). Memory Hierarchy (fastest·slowest, smallest·largest): CPU Registers (< 1ns, bytes) · L1 Cache (1-4ns,

Examples: (2 marks)

5-stage pipeline example: ADD R1,R2,R3 followed by SUB R4,R1,R5 causes RAW hazard on R1 · solved by forwarding from EX stage output back to EX input, eliminating 2-cycle stall. Cache hit ratio 95%, hit time 4ns, miss time 100ns: AMAT = 0.95×4

Conclusion: (1 mark)

Modern CPUs achieve high performance through deep pipelines (10-20 stages), out-of-order execution, superscalar issue (4-8 instructions/cycle), branch prediction (>95% accuracy), and multi-level caches. The memory hierarchy is carefully tuned to ensure most

Question 9: Web Technologies · Client-Server Architecture and HTTP [8 marks]

Introduction: (2 marks)

Web technologies encompass the protocols, languages, and frameworks used to build and deliver content over the World Wide Web. The client-server architecture is the foundational model of the web: a client (web browser or app) requests resources, and a server processes and responds. HTTP (HyperText Transfer Protocol), defined in RFC 2616 (HTTP/1.1) and RFC 7540 (HTTP/2), is the stateless application-layer protocol governing web

Core Concepts: (3 marks)

HTTP Request/Response: method (GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS), URL, headers (Content-Type, Authorization, Cache-Control, Accept), body (for POST/PUT). Status codes: 2xx Success (200 OK, 201 Created, 204 No Content), 3xx Redirection (301 Moved Permanently, 302 Found), 4xx Client Error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests), 5xx Server Error (500 Internal Server

Error, 503 Service Unavailable). HTTP/1.1 vs HTTP/2: HTTP/2 introduces multiplexing (multiple streams over one TCP connection, eliminating head-of-line blocking), header compression (HPACK), server push, binary framing (not text). HTTP/3 uses QUIC (UDP-based) for further latency reduction. REST (Representational State Transfer): stateless,

Examples: (2 marks)

GET /api/users/123 HTTP/1.1 · 200 OK with JSON body {id:123, name:'Alice'}. POST /api/login with {email, password} · server validates · returns JWT · client stores in localStorage · subsequent requests include Authorization: Bearer <token>. React SPA (Single Page Application): HTML shell loaded once; JavaScript fetches data via Fetch API /

Conclusion: (1 mark)

Modern web architecture has evolved from simple static pages to complex distributed systems with microservices, CDNs, and real-time communication. Understanding HTTP, REST, security, and client-server

Question 10: Cloud Computing · Service Models and Deployment [8 marks]

Introduction: (2 marks)

Cloud computing delivers computing resources (servers, storage, databases, networking, software, analytics) over the internet ('the cloud') on an on-demand, pay-as-you-go model, eliminating the need for organizations to own and maintain physical infrastructure. NIST (National Institute of Standards and Technology) defines cloud computing by five essential characteristics: on-demand self-service, broad network access, resource pooling (multi-tenancy), rapid elasticity (scale up/down automatically),

Core Concepts: (3 marks)

Service Models · IaaS (Infrastructure as a Service): provides virtualized compute, storage, and networking. Customer manages OS, middleware, apps. Providers: AWS EC2/S3, Azure VMs, Google Compute Engine. Use case: hosting custom applications, dev/test environments. PaaS (Platform as a Service): provides a development platform with runtime, middleware, OS managed by provider. Customer manages only applications and data. Providers: Google App Engine, Heroku, AWS Elastic Beanstalk, Azure App Service. Use case: web application development, APIs. SaaS (Software as a Service): fully managed application delivered over browser. Customer only uses the software. Providers: Google Workspace, Microsoft 365, Salesforce CRM, Slack, Zoom. Deployment Models · Public Cloud: shared infrastructure, owned by CSP (AWS, Azure, GCP), highly scalable, cost-effective. Private Cloud: dedicated infrastructure, hosted on-prem

Examples: (2 marks)

Netflix uses AWS IaaS (EC2, S3, CloudFront CDN) with multi-region deployment for 99.99% availability. A startup uses Heroku PaaS to deploy a Node.js API in minutes without managing servers. A hospital uses Private Cloud (on-prem VMware) for patient data compliance (HIPAA) with burst to AWS

Conclusion: (1 mark)

Cloud computing has democratized access to enterprise-grade infrastructure, enabling startups to compete with large enterprises. Choosing the right service model (IaaS/PaaS/SaaS) and deployment model (Public/Private/Hybrid) depends on cost, compliance, scalability needs, and technical capability.